

1. Introduction

- Project Title: **Agri-Tech**
- Team Members: **Hannah Bautista** - Full Stack Developer, Backend Developer, UI/UX Designer & QA

2. Project Overview

- Purpose: This Capstone project is a web-based agricultural technology (Agritech) platform designed to bridge the gap between traditional farming management and modern digital efficiency. It aims to empower local farmers and agricultural co-ops by providing tools for crop tracking, real-time inventory management, market pricing insights, and data-driven farming schedules to minimize waste and optimize yield output.

- Features:

Dynamic Dashboard: Real-time summary of active crop cycles, current storage status, and revenue analytics.

Crop Cycle Tracker: Enables farmers to log planting dates, monitor stages of growth, and schedule watering or fertilizer inputs.

Inventory & Produce Management: Track harvested crops, seeds, fertilizers, and equipment with low-stock alerts.

Marketplace / B2B Price Hub: Displays updated crop market trends or connects farmers directly with wholesalers.

Secure Authentication: Role-based access control (RBAC) separating administrative tasks (Co-op Managers/Admins) from standard data entry (Farmers/Field Workers).

3. Architecture

- Frontend

The frontend is constructed as a Single Page Application (SPA) using **React**. State management is handled through React Context API (or Redux Toolkit) to maintain user session states and global application data seamlessly. It features a component-based structure, client-side routing via React Router, and utilizes Axios for asynchronous HTTP requests to the backend server. The UI is fully responsive, leveraging CSS frameworks (such as Tailwind CSS or Material-UI) optimized for mobile viewports used in the field.

- Backend: The backend utilizes a RESTful API architecture built on **Node.js** and **Express.js**. It serves as the logical layer handling authentication middleware, data validation, and controller functions that communicate with the database. The server uses a decoupled controller-route pattern, ensuring clean separation of concerns, scalability, and robust error-handling layers.

- Database: Data persistence is handled by **MongoDB**, a NoSQL document database, coupled with **Mongoose** for object data modeling (ODM). Relationships between collections (such as linking a crop log to a specific user) are managed through Mongoose ObjectIds and referencing.

4. Setup Instructions

- Prerequisites:

- **Node.js** (v18.x or higher recommended)
- **npm** (v9.x or higher)
- **MongoDB Community Server** locally, or a cluster running on **MongoDB Atlas**
- **Git** for version control

- Installation:

1.Clone the Repository: Terminal Command.

Open your terminal and clone the project directory:

```
bash
```

```
git clone https://github.com/your-username/your-agritech-repo.git
```

```
git clone https://github.com/your-username/your-agritech-repo.git
```

```
cd your-agritech-repo
```

```
cd your-agritech-repo
```

```
cd client
```

```
cd client
```

```
npm install
```

```
npm install
```

Install Server Dependencies

```
bash
```

```
cd ../server
```

```
cd ../server
```

```
npm install
```

```
npm install
```

env

```
PORT=5000
```

```
PORT=5000
```

```
MONGO_URI=mongodb+srv://:@cluster0.example.mongodb.net/agritech
```

```
MONGO_URI=mongodb+srv://:@cluster0.example.mongodb.net/agritech
```

JWT_SECRET=your_super_secret_jwt_string_key

JWT_SECRET=your_super_secret_jwt_string_key

5. Folder Structure

your-agritech-project/

```
├── client/          # React Frontend
|   ├── public/      # Static public assets
|   └── src/
|       ├── assets/   # Project images and graphics
|       ├── components/ # Reusable UI components (Buttons, Cards, Navbar)
|       ├── context/   # Auth Context and Global State
|       ├── pages/     # Main views (Dashboard, CropTracker, Inventory, Login)
|       ├── services/  # Axios API instances and HTTP requests
|       ├── App.js     # Root Component & Client Routes
|       └── index.js   # React DOM Entrypoint
|
├── server/          # Node.js + Express Backend
|   ├── config/       # Database connection setups (db.js)
|   ├── controllers/   # Logical route handlers (authController, cropController)
|   ├── middleware/    # Custom middlewares (authMiddleware, errorMiddleware)
|   ├── models/        # Mongoose Database Schemas (User, Crop, Inventory)
|   ├── routes/        # Express API Route Definitions (api endpoints)
|   ├── .env           # Private Environment variables (Local only)
|   └── server.js      # Server entry point
└── README.md
```

6. Running the Application

Running the Frontend

1. Open a terminal tab and navigate to the client folder:

Bash

```
cd client
```

2. Start the React development server:

```
```bash
```

```
npm start
```

### **Running the Backend**

1. Open a separate terminal tab and navigate to the server folder:

Bash

```
cd server
```

2. Start the Express development environment:

```
```bash
```

```
npm start
```

7. API Documentation

Authentication Routes

POST /api/auth/register - Registers a new farming user account.

POST /api/auth/login - Authenticates user credentials and issues a Token.

Crop Logging & Management Routes

GET /api/crops - Fetches all active crop data logs for the authenticated user.

POST /api/crops - Creates a new tracking log entry for a newly planted crop field.

Example API Request/Response Data

POST /api/crops

Headers Required: Authorization: Bearer <JWT_TOKEN>

8. Authentication

1. The system secures private data using a statutory stateless authentication mechanism: JSON Web Tokens (JWT).

2. **Login Verification:** When a user enters their credentials, the server validates the user email and verifies the password using bcryptjs hashing.
3. **Token Issuance:** Upon verification, the backend generates a signed JWT payload containing the user ID and application role.
4. **Client Storage:** The frontend receives this token and safely holds it within memory state/browser localStorage.
5. **Authorized Middleware Access:** For any protected action (e.g., updating crop logs), the client attaches the token to the HTTP header as an authorization attribute. The backend route processes this token via custom authorization middleware (authMiddleware.js) prior to returning any sensitive response data.

9. User Interface

- **Authentication Portal:** Clean, minimalistic landing login screen displaying accessible password recovery options.
- **Dashboard Tracker Overview:** Visual display indicators emphasizing active fields, expected harvests, and real-time inventory levels.
- **Form Analytics View:** Input interfaces optimizing fast, straightforward single-form inputs for field operations tracking

10. Testing

- **API Verification:** Handled using tools like **Postman** to run integration tests across all Express routes, verifying response bodies, payload compliance, and HTTP status handling.
- **Component and UI Verification:** Manual end-to-end interface checks monitoring component responsive breakpoints, visual continuity, form submission errors, and input validations across different screen sizes.

11. Screenshots or Demo

https://drive.google.com/drive/folders/1Oo5yUU-WYbBkztBjV01LLi7BmUa9hPB4?usp=drive_link

12. Known Issues

- **Token Expiry Reactivity:** If an authorized session token expires while a user fills out a multi-page crop tracking form, progress may clear on submit without saving data. Catching this state error cleanly is currently under refinement.
- **Offline Support Boundary:** Form components require active data connections to save log attributes. Submissions in areas with zero network connectivity will fail to sync until the system re-establishes a live backend handshake.

13. Future Enhancements

- **IoT Integration Integration:** Linking hardware soil moisture sensors directly into the Express backend API, populating automated watering metrics to dashboard data blocks.
- **Localized SMS Notifications:** Integrating third-party APIs (like Twilio) to broadcast automated SMS weather warnings or critical harvesting reminders directly to farmers without smartphones.
- **Offline-First Sync Engine:** Incorporating service workers and IndexedDB setups to allow field data logging when completely disconnected, uploading queued records automatically once internet service returns.